

Peer-Review 1: UML

Stefano Castiglione, Lorenzo Bressi, Davide Bombelli, Nicolò Sardella

Gruppo 30

Valutazione del diagramma UML delle classi del gruppo 39.

Lati positivi

- Il diagramma è completo di tutte le funzionalità necessarie allo svolgimento del gioco.
- La presenza di commenti ha agevolato la comprensione.
- Tutte le relazioni tra le classi sono rappresentate.

Lati negativi

- Gestione del punteggio poco chiara: ridondanza tra l'attributo di tipo `int` presente all'interno di `Player` e `ScoreTracker` organizzato come lista;
- Elementi e Risorse sono differenziati in `Player` e in `Objectives`, ma non nell'enumerazione `ResourceElement`;
- `addResourceMana` / `addElementsMana`: vista la scelta di rappresentare le risorse/elementi con `enum ResourceElement`, sarebbe più coerente avere come parametro di queste funzioni un `ResourceElement` e salvare il conteggio in un attributo di tipo `Map<ResourceElement,Integer>` oppure `MultiSet<ResourceElement>`, al posto di avere 2 array di interi;
- Nome delle carte `NonObjectiveCard` poco esplicativo: potrebbe essere più opportuno chiamarle, per esempio, `PlaceableCards` (oppure distinguere completamente carte e obiettivi);
- `setCardSlot()` dovrebbe avere la carta da piazzare come parametro. Inoltre si può omettere il suffisso "slot" nei metodi all'interno di `Slot` poiché ridondante;
- In `ObjectiveCardEffect` non ci sono scritti i metodi all'interno dell'interfaccia, e potrebbe essere più lineare includerli direttamente in `ObjectiveCard`;
- La relazione di ereditarietà delle carte può essere resa più chiara e leggibile:
 - i. Separando completamente le carte piazzabili dalle carte obiettivo
 - ii. Mantenendo una sola classe per ogni tipo di carta (magari differenziando sottotipi di carte che non hanno grosse differenze solo con un attributo di tipo `enum`, come ad esempio nel caso delle `ResourceGoldCard`);

- `resourceElementSet: ArrayList<ResourceElement>` : il nome dell'attributo non è coerente con il tipo di oggetto;
- manca la rappresentazione degli obiettivi basati sul posizionamento delle carte in `Field` ;
- Sarebbe opportuno snellire eliminando metodi di tipo getter e setter superflui o unendone alcuni. Di seguito qualche esempio:
 - `setPoints()` : il setter è superfluo perché i punti sono impostati a priori e mai più modificabili, quindi avrebbe più senso inizializzare l'attributo nel costruttore senza rivedere possibili modifiche successive;
 - `getFrontCorners()` / `getBackCorners()` : si potrebbero unire in un unico `getVisibleCorners()` che in base al valore di `isPlacedFront` sceglie quale array ritornare

Problemi formali:

- Simbolo dell'ereditarietà errato (freccia piena al posto di freccia vuota);
- Mancano i nomi dei parametri dei metodi (è presente solo il tipo, che però non sempre è sufficientemente descrittivo). La sintassi corretta è `metodo(argN: TipoArgN): TipoDiRitorno` ;
- Simbolo $\oplus$: non è chiaro l'utilizzo della relazione di containment;
- Classi astratte (come `ObjectiveCard`) dovrebbero essere rappresentate con il nome della classe in corsivo;
- La grande quantità di classi e composizioni rende il diagramma visivamente caotico, potrebbe essere opportuno riordinarle o evitare di rappresentare qualche relazione meno rilevante.

Confronto tra le architetture

La nostra architettura è più concentrata sulla leggibilità e sulla rappresentazione astratta, mentre la loro scende più nel dettaglio. A seguito della revisione, ci siamo accorti di poter esplicitare con più chiarezza:

- la gestione del primo turno;
- la gestione della scelta del token da parte del giocatore;
- commenti associati alle classi di interpretazione meno ovvia.